

I. Abstract

For the final project we were tasked to create a multilayered perceptron (MLP), which is a type of neural network that consists of several layers, capable of classifying handwritten digits on 28x28 grayscale images. The eventual goal was to reliably and consistently classify digits of the class dataset after training with the MNIST dataset. For developing the neural network we use a fine-tuning approach where we select one independent variable and conduct several tests with different values. The test that resulted in the best accuracy scores we use in later stages of testing. The best accuracy scores we managed to achieve with this approach is ~62% on the class data and ~97% on the MNIST data.

II. High Level Framework

After many hours of training and testing with different configurations of adjusting the hyperparameters and architecture, the best results came from this configuration...

Hyperparameters:

Epochs: 30

Batch Size: 128

Learning Rate: 0.001

Architecture:

Input Layer: 784 neurons

H1: 512 neurons

H2: 128 neurons

H3: 64 neurons

Output Layer: 10 logits

The total number of trainable parameters in the model is 476,490 weights and biases. Training was performed on 48,000 images (80% of the 60,000 image MNIST training set) with the remaining 12,000 used for validation. At a batch size of 128 over 30 epochs, the model completed 11,250 total training iterations.

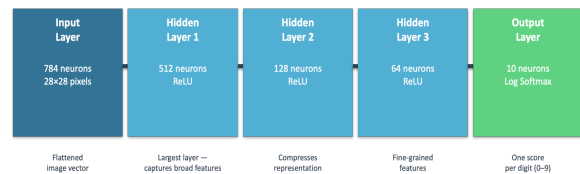


Figure 1: Visualization of model's architecture.

III. Solution In Detail

To start off we constructed the training and validation sets using an 80/20 split on the MNIST dataset.

```
trainset = datasets.MNIST('~/.pytorch/MNIST_data/', download=True, train=True, transform=transform)

# Split training set into 80% train and 20% validation
train_size = int(0.8 * len(trainset))
val_size = len(trainset) - train_size
train_data, val_data = torch.utils.data.random_split(trainset, [train_size, val_size])
```

We follow the neural network class implementation shared in the lecture slides which defines our layers and activation function.

```
# class structure for nn
class Network(nn.Module):
    # making of network
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784,512)
        self.fc2 = nn.Linear(512,128)
        self.fc3 = nn.Linear(128,64)
        self.output = nn.Linear(64,10)

    # feedforward process
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        x = F.log_softmax(self.output(x), dim=1)
        return x
```

For our hyperparameters we define our batch sizes within the data loaders and our

learning rate within our optimizer. Epochs are defined separately...

```
# Create data loaders
trainloader = torch.utils.data.DataLoader(train_data, batch_size=128, shuffle=True)
valloader = torch.utils.data.DataLoader(val_data, batch_size=128, shuffle=False)
testloader = torch.utils.data.DataLoader(testset, batch_size=128, shuffle=False)

epochs = 30
model = Network()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.NLLLoss()
```

For our training loop we go through 3 phases. First, we train on all the images in the training dataloader using the same training pipeline as shown in figure 2. Then we validate our model's accuracy using unseen images. In the last phase, we compute the average training loss and validation loss during that epoch and print it so we can track the progression of the program while it is executing. Those values will be used later for plotting and analysis.

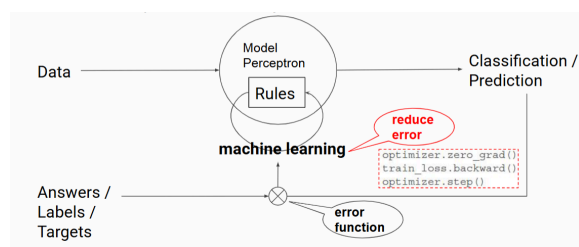


Figure 2: training pipeline model from class slides.

```
for epoch in range(epochs):
    # Training phase
    model.train()
    running_train_loss = 0.0
    for images, labels in trainloader:
        images = images.view(images.shape[0], -1).to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        output = model(images)
        loss = loss_fn(output, labels)
        loss.backward()
        optimizer.step()
        running_train_loss += loss.item()

    # Validation phase (after each epoch)
    model.eval()
    running_val_loss = 0.0
    with torch.no_grad():
        for images, labels in valloader:
            images = images.view(images.shape[0], -1).to(device)
            labels = labels.to(device)
            output = model(images)
            loss = loss_fn(output, labels)
            running_val_loss += loss.item()

    # Record and print epoch results
    avg_train_loss = running_train_loss / len(trainloader)
    avg_val_loss = running_val_loss / len(valloader)
    train_loss.append(avg_train_loss)
    val_loss.append(avg_val_loss)
    print(f'Epoch {epoch+1}/{epochs} - Train Loss: {avg_train_loss:.4f} - Val Loss: {avg_val_loss:.4f}')
```

Here we plot our training loss and validation loss from the information we gathered during the training loop.

```
# Plot training and validation loss per epoch
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```

Here's our function to interpret the images from the class data. We "sanitize" the data by converting to grayscale by opening the image and using the convert function. Then we use the resize function to fix incorrectly sized images. This is so that we can make a dataloader and verify against the data later.

```
# This function takes an image name and returns the data as an ndarray
def ProjectDataLoader(folder='./Handwritten-Digits-Spring-2026/digits'):
    images, labels = [], []
    for fname in sorted(os.listdir(folder)):
        if not fname.endswith('.png'):
            continue
        try:
            label = int(fname.split('-')[0])
        except ValueError:
            continue
        img = Image.open(os.path.join(folder, fname)).convert('L')
        img = img.resize((28, 28), Image.LANCZOS)
        images.append(np.array(img))
        labels.append(label)
    print(f'Loaded {len(images)} group images')
    return np.array(images), np.array(labels)

group_images, group_labels = ProjectDataLoader()
```

We call ProjectDataLoader() to load all class handwritten digit images from the GitHub repository. The function reads each PNG file, converts it to grayscale, resizes it to 28x28, and returns the images and their labels as numpy arrays parsed from the filename format digit-groupID-memberID.png.

IV. Test Results and Analysis

After training the final model with the selected hyperparameters (Adam, NLLoss, lr=0.001, batch=128, 30 epochs, 784→512→128→64→10), we evaluated its performance on the MNIST test set and the class handwritten digit dataset. We present results starting with our architecture testing that led to the final configuration, followed by the full MNIST and group data results

Architecture Testing

Before settling on our final architecture, we trained and evaluated three MLP configurations to compare how layer depth and width affect both MNIST accuracy and generalization to the class handwriting dataset. All three were trained with the same settings (Adam, NLLoss, lr=0.001, batch=64, 20 epochs) so the only variable was architecture.

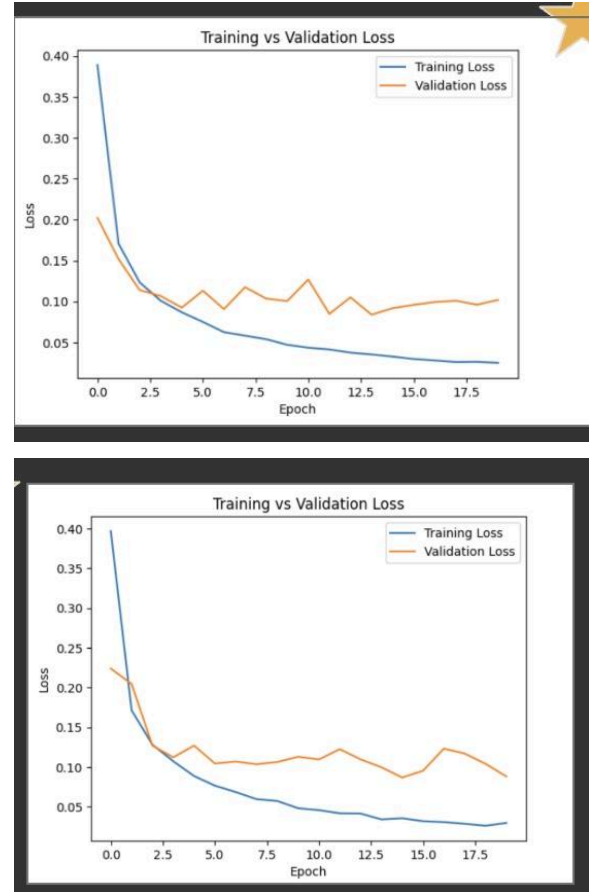
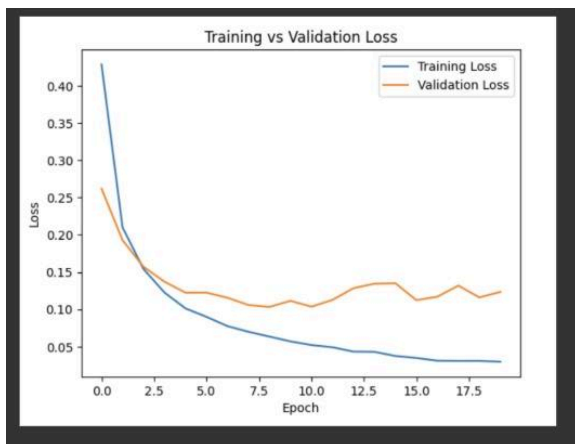


Fig. 2:

Training vs Validation Loss for all three architectures. Left: 784→128→64→10 (2 hidden layers). Center: 784→512→128→64→10 (3 hidden layers, selected ★). Right: 784→512→256→128→64→10 (4 hidden layers). The center model shows the best balance - validation loss converges close to training loss without the wide gap seen in the 2-layer model.

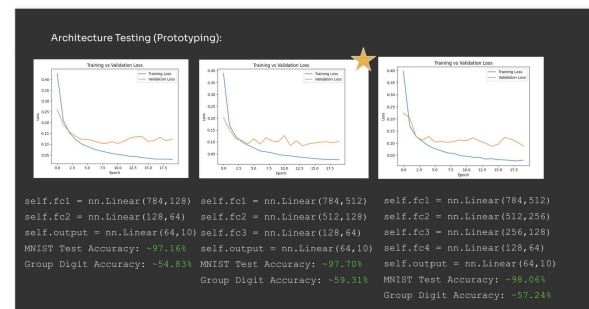


Table 1: Architecture comparison (Adam + NLLoss, 20 epochs, lr=0.001, batch=64):

Architecture	MNIST Acc	Group Acc	Notes
784→128→64→10 (2 hidden)	97.16 %	54.83 %	Underfit - too small
784→512→128→64→10 (3 hidden)	97.70 %	59.31 %	← Selected
784→512→256→128→64→10 (4 hidden)	98.06 %	57.24 %	Higher MNIST, lower group accuracy

The 3-hidden-layer model (784→512→128→64→10) gave the best group accuracy at 59.31% with strong MNIST accuracy at 97.70%. We selected this architecture for all subsequent hyperparameter testing.

6.2 Hyperparameter Testing

With the architecture fixed, we tested each remaining hyperparameter independently using the same one variable change at a time approach, fix everything else and sweep one variable and pick the best result. Then move on to the next. We tested learning rates, batch size, and epoch count

Optimizer + Loss Functions

Before tuning hyperparameters, we tested four combinations of optimizer and loss function with all other settings fixed (20 epochs, lr=0.001, batch=64) to establish which pairing gave the best starting point. Adam with NLLoss outperformed all other

combinations with 97.68% MNIST accuracy and 54.83% group accuracy - a 10 percentage point improvement over the SGD baseline on class data. SGD with CrossEntropy showed marginal improvement over the SGD baseline but still significantly underperformed Adam. We fixed Adam + NLLoss for all subsequent testing.

Optimizer	Loss	MNIST Acc	Group Acc	Notes
SGD	NLLoss	96.86 %	44.14 %	Baseline
SGD	CrossEntropy	96.63 %	46.21 %	
Adam	NLLoss	97.68 %	54.83 %	← Selected
Adam	CrossEntropy	96.91 %	51.38 %	

Learning Rate

We compared five learning rates (0.0001, 0.0005, 0.001, 0.0015, 0.01) with all other settings fixed at 30 epochs, batch=64.

Independent Variable = LR				
Test 1: Epochs = 30 Batch size = 64 LR = 0.0001 MNIST Test Accuracy: 97.99% Group Digit Accuracy: 53.45%	Test 2: Epochs = 30 Batch size = 64 LR = 0.0005 MNIST Test Accuracy: 97.56% Group Digit Accuracy: 55.86%	Test 3: ★ Epochs = 30 Batch size = 64 LR = 0.001 MNIST Test Accuracy: 97.89% Group Digit Accuracy: 57.93%	Test 4: Epochs = 30 Batch size = 64 LR = 0.0015 MNIST Test Accuracy: 97.69% Group Digit Accuracy: 56.90%	Test 5: Epochs = 30 Batch size = 64 LR = 0.01 MNIST Test Accuracy: 95.19% Group Digit Accuracy: 47.93%

Fig. 4:

Learning rate sweep results. lr=0.001 (Test 3, marked ★) achieved the best group digit accuracy at 57.93%. lr=0.01 produced the worst results (47.93% group, 95.19% MNIST) due to unstable training. lr=0.0001 converged too slowly and plateaued prematurely at 53.45% group accuracy.

Table 2: Learning rate comparison (Adam, NLLoss, 30 epochs, batch=64):

LR	Epoch	Batch	MNIST Acc	Group Acc	Notes
0.0001	30	64	97.95%	53.45%	Converges slowly, plateaus early
0.0005	30	64	97.56%	55.86%	
0.001	30	64	97.89%	57.93%	← Selected: smoothest curve
0.0015	30	64	97.65%	56.90%	
0.01	30	64	95.19%	47.93%	Unstable, fluctuating loss

lr=0.001 is the standard Adam default and gave the smoothest training curve with the best group accuracy. Higher rates caused loss instability and lower accuracy on both datasets. Lower rates converged too slowly within 30 epochs. We fixed lr=0.001 for all subsequent tests.

Batch Size

We compared three batch sizes (32, 64, 128) with all other settings fixed at lr=0.001, 30 epochs.

Independent Variable = Batch Size		
Test 1: Epochs = 30 Batch size = 32 LR = 0.001 MNIST Test Accuracy: ~ 97.36% Group Digit Accuracy: ~ 59.31%	Test 2: Epochs = 30 Batch size = 64 LR = 0.001 MNIST Test Accuracy: ~ 97.69% Group Digit Accuracy: ~ 55.53%	Test 3: Epochs = 30 Batch size = 128 LR = 0.001 MNIST Test Accuracy: ~ 97.80% Group Digit Accuracy: ~ 61.72%

Fig. 5:

Batch size sweep results. Batch size 128 (Test 3, marked ★) achieved the highest group digit accuracy at 61.72% and the highest MNIST accuracy at 97.80%. Batch size 64 gave lower group accuracy (55.53%) despite similar MNIST accuracy, and batch 32 scored 59.31% on group data.

Table 3: Batch size comparison (Adam, NLLoss, 30 epochs, lr=0.001):

Batch	Epoch	LR	MNIST Acc	Group Acc	Notes
32	30	0.001	97.36%	59.31%	
64	30	0.001	97.69%	55.53%	
128	30	0.001	97.80%	61.72%	← Selected: best group accuracy

Larger batch sizes gave the optimizer smoother gradient estimates, which appeared to help generalization of the class handwriting dataset. Batch 128 gave the best performance on both datasets. We fixed batch=128 for epoch testing.

Epoch Count

We compared five epoch counts (10, 20, 30, 40, 50) with all other settings fixed at lr=0.001, batch=128.

Independent Variable = Epochs				
Test 1: Epochs = 10 Batch size = 128 LR = 0.001 MNIST Test Accuracy: ~ 97.39% Group Digit Accuracy: ~ 56.21%	Test 2: Epochs = 20 Batch size = 128 LR = 0.001 MNIST Test Accuracy: ~ 97.72% Group Digit Accuracy: ~ 56.59%	Test 3: ★ Epochs = 30 Batch size = 128 LR = 0.001 MNIST Test Accuracy: ~ 97.87% Group Digit Accuracy: ~ 61.38%	Test 4: Epochs = 40 Batch size = 128 LR = 0.001 MNIST Test Accuracy: ~ 97.85% Group Digit Accuracy: ~ 64.83%	Test 5: Epochs = 50 Batch size = 128 LR = 0.001 MNIST Test Accuracy: ~ 97.69% Group Digit Accuracy: ~ 67.24%

Fig. 6:

Epoch count sweep results. 30 epochs (Test 3, marked ★) achieved the best group digit accuracy at 61.38% and strong MNIST accuracy at 97.87%. At 40 epochs

group accuracy dropped sharply to 54.83%, indicating the model began overfitting to MNIST's distribution. 50 epochs partially recovered to 57.24% but still underperformed 30 epochs on group data.

Table 4: Epoch count comparison (Adam, NLLoss, batch=128, lr=0.001):

Epochs	Batch	LR	MNIST Acc	Group Acc	Notes
10	128	0.001	97.39%	56.21%	Underfit
20	128	0.001	97.73%	56.55%	
30	128	0.001	97.87%	61.38%	← Selected: peak group accuracy
40	128	0.001	97.57%	54.83%	Group accuracy drops - overfitting MNIST
50	128	0.001	97.69%	57.24%	

Group accuracy peaked at 30 epochs (61.38%) then declined sharply at 40 epochs (54.83%) a clear sign the model was overfitting to MNIST's distribution beyond this point. The training vs validation loss plot (Fig. 3 in Section 3.3) confirms loss converges stably at 30 epochs without a widening gap. We therefore used 30 epochs for all final model evaluation.

6.3 MNIST Test Set Results

MNIST Test Accuracy: 97.78% Error Rate: 2.22% Dataset: 10,000 images

The MNIST test set is our baseline - the model was trained on MNIST so strong performance here confirms the architecture and training procedure are effective. The confusion matrix shows that almost all digits are classified correctly, with occasional errors in visually similar pairs. The per-digit bar chart highlights which digits fell below 98% accuracy.

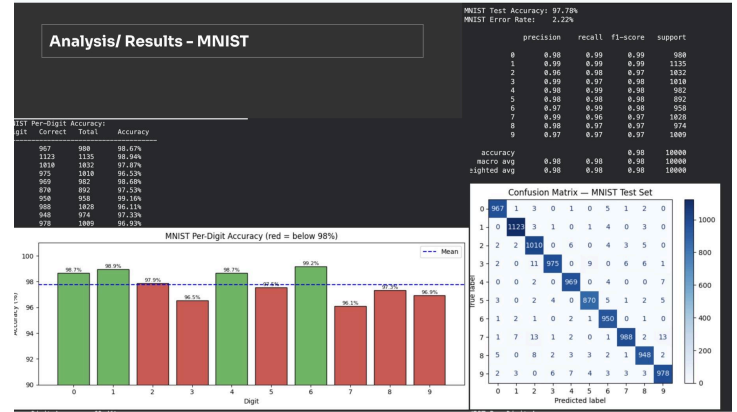


Fig. 7:

Analysis/Results MNIST. Top right: overall accuracy (97.78%), error rate (2.22%), and classification report showing precision, recall, and F1-score per digit. All digits score between 0.96–0.99 across all metrics. Bottom left: per-digit accuracy bar chart green bars are at or above 98%, red bars are below. Digit 6 was strongest at 99.16%; digits 3 and 7 were weakest at 96.53% and 96.11%. Bottom right: confusion matrix dense diagonal confirms nearly all predictions are correct. Main off-diagonal errors are 3 confused with 8, 7 confused with 9, and 4 confused with 9.

Per-digit results confirm that the model learned digit structure well across all classes. Digits 3 and 7 are weakest digit 3 is frequently confused with 8 and digit 7 with 9. These are expected MLP failure modes: without spatial awareness, globally similar pixel distributions produce similar output vectors.

6.4 Class Handwritten Dataset Results

Group Digit Accuracy: 62.41% Error Rate: 37.59% Dataset: 290 images (29 per digit)

The class dataset is significantly more challenging than MNIST due to variation in writing style, stroke thickness, slant, size, and image quality across different students. Accuracy dropped approximately 35 percentage points from MNIST performance expected given the model was trained exclusively on standardized MNIST digits and has never seen the diversity of our class handwriting.

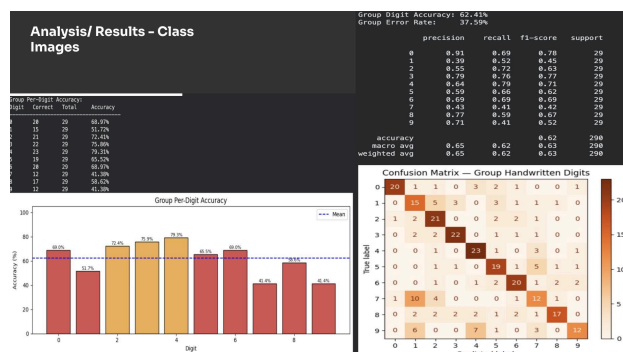


Fig. 8:

Analysis/Results Class Images. Top right: overall accuracy (62.41%), error rate (37.59%), and classification report. Digit 1 has the lowest F1-score (0.45) and digit 0 has the highest precision (0.91). Bottom left: per-digit accuracy bar chart with mean line digits 2, 3, 4 are above the mean while digits 1, 7, 9 are well below. Bottom right: confusion matrix for group handwritten digits notably more off-diagonal mass than the MNIST matrix. Row 7 shows 10 sevens predicted as 1, and row 9 shows 7 nines predicted as 4.

The per-digit results reveal specific failure patterns. Digit 4 was strongest at 79.3% while digits 7 and 9 were worst at 41.4% each less than half. The confusion matrix exposes why: 10 of 29 sevens were predicted as 1. A handwritten 7 without a

clear crossbar shares a dominant vertical stroke with 1, and since the MLP flattens the image to a 784-element vector with no spatial awareness, the positional similarity triggers misclassification. Similarly, 7 of 29 nines were predicted as 4 the closed upper loop of a 9 can resemble the upper half of a 4 when the lower tail is thin. These are systematic failures driven by the MLP's inability to detect local spatial features.

Analysis: What Works, What Doesn't, and Why

What works: The model generalizes strongly within the MNIST distribution with 97.78% accuracy with consistent precision and recall across all digit classes. Simple digits like 0, 1, and 4 score near-perfectly. The Adam optimizer with NLLoss trained stably and converged without overfitting at 30 epochs.

What doesn't: Performance drops 35 points on the class dataset. Digits 7 and 9 score only 41.4% worse than random guessing for a 10-class problem would be expected at 10%. Specific group writing styles cause the model to completely fail on some subsets (Group 1: 25%).

Why: The core reason is the MLP's spatial blindness. Flattening a 28×28 image to a 784-element vector destroys all positional information. A digit written in a slightly different position, at a different scale, or with a different stroke style produces a completely different input vector even if it is visually the same digit to a human. MNIST digits are centered, normalized, and consistently proportioned. Our class digits are not.

V. Future Work

One area we would like to explore in future work is implementing dropout regularization. Dropout works by randomly deactivating a percentage of neurons during each training pass, which forces the network to learn more generalized representations rather than memorizing patterns specific to the training data. We believe this could help reduce the gap between our MNIST accuracy and our class dataset accuracy by making the model less sensitive to the specific pixel distributions in MNIST. Different dropout rates would need to be tested systematically using the same one variable at a time approach we applied throughout this project to find the configuration that best improves generalization without significantly hurting MNIST performance.

Another direction worth exploring would be data augmentation on the MNIST training set, such as small random rotations, shifts, and scaling. Since the primary reason our model struggles on class data is the variation in how people write, training on a more varied dataset could help the model learn more robust representations. This would not require changing the MLP architecture at all and could be implemented directly through PyTorch's transform pipeline.

VI. Conclusion

In this project we built and trained a multilayer perceptron to classify handwritten digits on 28x28 grayscale images. Through systematic hyperparameter tuning where we tested optimizers, loss functions,

architectures, learning rates, batch sizes, and epoch counts one variable at a time, we were able to improve our group digit accuracy from a baseline of 44.14% using SGD to a final result of 62.41% using Adam with NLLoss. Our final model achieved 97.78% accuracy on the MNIST test set and 62.41% on the class handwritten digit dataset using an architecture of 784 to 512 to 128 to 64 to 10 neurons trained over 30 epochs with a batch size of 128 and learning rate of 0.001.

The most significant challenge we encountered was the gap between MNIST performance and real world handwriting performance. The core reason for this gap is that flattening a 28x28 image into a 784 element vector removes all positional information from the image. This means that a digit written at a slightly different position, scale, or stroke style produces a completely different input even if it looks the same to a human. MNIST digits are carefully centered and normalized while our class dataset reflects the natural variation in how different people write. This is a fundamental limitation of the MLP architecture that a convolutional neural network would handle much better since it preserves spatial relationships between pixels.

Overall this project gave us hands on experience with the full machine learning workflow from data loading and preprocessing through model design, training, validation, and evaluation. We came away with a much stronger understanding of how hyperparameter choices interact and why no single parameter can be tuned in isolation.

VII. Github Repository

https://github.com/jdaltongit/CSCI580_Spring26_Group7

VIII. References

“How to Normalize Images in Pytorch ?”
GeeksforGeeks, GeeksforGeeks, 6 June 2022,
www.geeksforgeeks.org/python/how-to-normalize-images-in-pytorch/.

“MNIST : Tensorflow Datasets.”
TensorFlow,
www.tensorflow.org/datasets/catalog/mnist.
Accessed 14 May 2026.

“Normalize.” Normalize - Torchvision
Main Documentation,
docs.pytorch.org/vision/main/generated/torchvision.transforms.Normalize.html.
Accessed 14 May 2026.

Shen, Bo. “BuildNN-1.Ipynb.”

Shen, Bo.
“LoadImage-1.Ipynb.Mnist.Png.Zip.”

Shen, Bo. “Spring26 CSCI580 Week13.1-2 -
MLP+BackProp.Pdf.”

Shen, Bo. “Spring26 CSCI580 Week13.3 -
BuildTrainNN.Pdf.”

Shen, Bo. “Spring26 CSCI580 Week14 -
ModelTuning.Pdf.”

Shen, Bo. “Spring26 CSCI580 Week15.1 -
PerfEval.Pdf.”

“ToTensor.” ToTensor - Torchvision Main
Documentation,
docs.pytorch.org/vision/main/generated/torchvision.transforms.ToTensor.html. Accessed
14 May 2026.